

BCC202 - Estruturas de Dados I

Aula 12: Pilhas

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Introdução

TAD Pilha

Implementação com Vetor

Implementação via Ponteiros

Considerações Finais

Bibliografia

Introdução

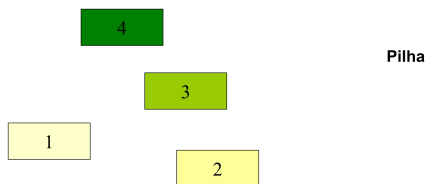
Tipo Abstrato de Dados com as seguintes características:

- ▶ O **último** elemento a ser inserido é o **primeiro** a ser retirado/removido.
- ▶ **LIFO - Last in First Out.**
- ▶ TAD conhecida como stack.

Cada novo elemento é inserido no topo e só temos acesso ao elemento do topo.

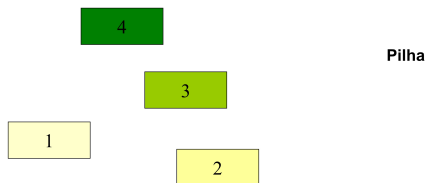
- ▶ **Analogia** natural com o conceito de **pilhas** de pratos que usamos no **dia a dia**:
 - ▶ Pilha de pratos.
 - ▶ Pilha de livros.
- ▶ Usos:
 - ▶ Chamada de subprogramas, avaliação de expressões aritméticas, etc.

Funcionamento de uma Pilha



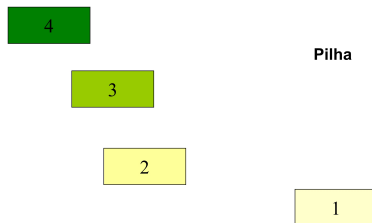
Pilha vazia.

Funcionamento de uma Pilha



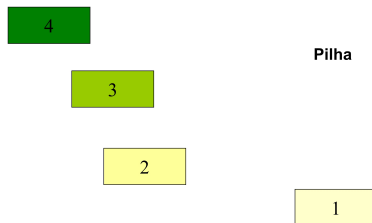
Empilhar.

Funcionamento de uma Pilha



Empilhar.

Funcionamento de uma Pilha



Empilhar.

Funcionamento de uma Pilha

4

3

Pilha

2
1

Empilhar.

Funcionamento de uma Pilha

4

3

Pilha

2

1

Empilhar.

Funcionamento de uma Pilha

4

Pilha

3
2
1

Empilhar.

Funcionamento de uma Pilha

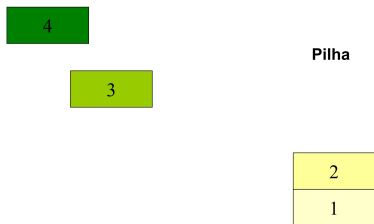
4

Pilha

3
2
1

Desempilhar.

Funcionamento de uma Pilha



Desempilhar.

Funcionamento de uma Pilha

4

3

Pilha

2

1

Empilhar.

Funcionamento de uma Pilha

4

Pilha

3

2

1

Empilhar.

Funcionamento de uma Pilha

4

Pilha

3

2

1

Empilhar.

Funcionamento de uma Pilha

Pilha

4
3
2
1

Empilhar.

Funcionamento de uma Pilha

Pilha

4
3
2
1

Desempilhar.

Funcionamento de uma Pilha

4

Pilha

3
2
1

Desempilhar.

Funcionamento de uma Pilha

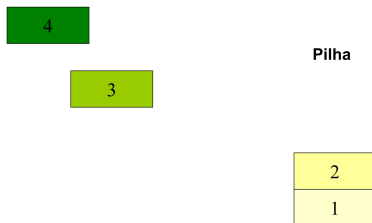
4

Pilha

3
2
1

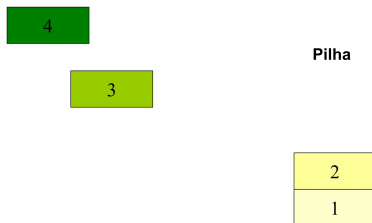
Desempilhar.

Funcionamento de uma Pilha



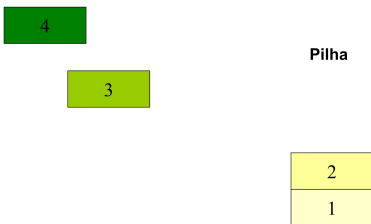
Desempilhar.

Funcionamento de uma Pilha



Pilha nada mais é do que uma **Lista** com restrições de inserção e remoção:

Funcionamento de uma Pilha



Pilha nada mais é do que uma **Lista** com restrições de inserção e remoção:

O **último** elemento a ser inserido é o **primeiro** a ser retirado.

Exemplo: Pilha de Execução

Pilha de execução da linguagem C.

- ▶ As variáveis locais são dispostas numa pilha, a **pilha de execução**.
- ▶ Uma função só tem acesso às variáveis que estão no topo.
- ▶ Não é possível acessar variáveis locais de outras funções.

Exemplo: Pilha de Execução

Pilha de execução da linguagem C.

- ▶ As variáveis locais são dispostas numa pilha, a **pilha de execução**.
- ▶ Uma função só tem acesso às variáveis que estão no topo.
- ▶ Não é possível acessar variáveis locais de outras funções.

Exemplo: Pilha de Execução

Pilha de execução da linguagem C.

- ▶ As variáveis locais são dispostas numa pilha, a **pilha de execução**.
- ▶ Uma função só tem acesso às variáveis que estão no topo.
- ▶ Não é possível acessar variáveis locais de outras funções.

TAD Pilha

pilha.h

```
1 #ifndef pilha_h
2 #define pilha_h
3
4 /* definindo um novo tipo */
5 typedef ... Pilha;
6
7 Pilha* PilhaInicia();
8 int PilhaEhVazia(Pilha*);
9 int PilhaPush(Pilha*, int); /* insere no mesmo lugar que remove */
10 int PilhaPop(Pilha*, int*); /* retira do mesmo lugar que insere */
11 void PilhaLibera(Pilha**);
12
13 #endif /* pilha_h */
```

Existem várias opções de estruturas de dados que podem ser usadas para representar pilhas. As duas representações mais utilizadas são: **vetor** e **ponteiros**.

Implementação com Vetor

struct

Devemos ter um vetor para armazenar os elementos da pilha.

- ▶ um novo elemento é armazenado na primeira posição vazia do vetor (posição vazia de menor índice).
- ▶ a pilha pode ter um limite conhecido (vetor estático): é necessário verificar a sua capacidade a cada inserção.
- ▶ a pilha pode usar um vetor dinâmico, com a dimensão atualizada (via uso de *realloc*) sempre que necessário.

Struct Pilha utilizando Vetores

```
1 #define MAXTAM 1000 /* verificar capacidade a cada inserção */
2
3 typedef struct{
4     int n; /* número de elementos armazenados */
5     int vet[MAXTAM]; /* alocação estática */
6 } Pilha;
7
8 /* implementar as funções de pilha.h para manipular os dados especificados */
```

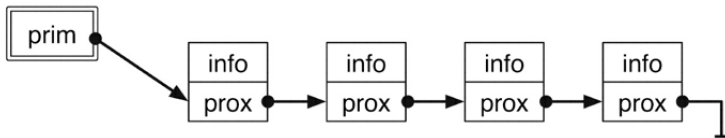
Implementação via Ponteiros

Ponteiros

A implementação será similar à lista encadeada.

- ▶ Elementos serão inseridos e retirados da **primeira** posição da pilha (**topo**).
- ▶ Será mantido um **ponteiro**: **prim** ou **topo**.

prim é um ponteiro que corresponde ao topo da pilha.



E se mantivéssemos como topo a última posição?

O mesmo que a *Lista*: nós encadeados com informações e ponteiro para o próximo nó.

`pilha_ponteiro.c`

```
1 #ifndef pilha_h
2 #define pilha_h
3
4 typedef struct {
5     int chave;
6 } Item;
7
8 /* Decisão de implementação, poderia ser vetor */
9 typedef struct cel {
10     Item item;
11     struct cel *prox;
12 } Celula;
13
14 typedef struct {
15     TCelula *cabeca;
16 } Pilha;
17
18 /* implementar as funções de "pilha.h" */
```

Considerações Finais

- ▶ Pilha: Tipo Abstrato de Dado.
 - ▶ Vetor.
 - ▶ Ponteiro.
- ▶ **Protocolo bem definido:** (Last In, First Out - LIFO).
- ▶ Qual a complexidade de tempo para empilhar (*push*) e para desempilhar (*pop*)?
- ▶ Link de um exemplo de implementação.

TAD Fila.

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados do livro-texto indicado a seguir:



Celes, Waldemar and Cerqueira, Renato and Rangel, José

Introdução a Estruturas de Dados com Técnicas de Programação em C.

Elsevier Brasil, 2016.

ISBN 978-85-352-8345-7.

Exercício

Seja Lista o TAD Lista que implementa as seguintes funções:

```
1 /* faz com que a lista fique vazia */
2 void ListaInicia(Lista *pLista);
3
4 /* insere pItem na posicao pos da lista */
5 void ListaInsere(Lista *pLista, Item* pItem, int pos);
6
7 /* retira o item na posicao pos da lista e retorna em pItem */
8 int ListaRetira(Lista* pLista, Item* pItem, int pos);
9
10 int ListaTamanho(Lista *lista);
```

Implemente as operações abaixo do TAD Pilha utilizando Lista. Esta implementação é eficiente? Justifique sua resposta.

```
1 Pilha* PilhaInicia();
2 int PilhaEhVazia(Pilha*);
3 int PilhaPush(Pilha*, int); /* insere no inicio */
4 int PilhaPop(Pilha*, int*); /* retira do inicio */
```